

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2.5

Student Name: J.Sravan Kumar

Roll No.: A24126552085

Date: 25-08-2026

1. TITLE

Develop a Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To develop a mini web application using **JavaScript** that accepts student information through an HTML form, creates a **Student object using a JavaScript class**, and dynamically displays the submitted details using **DOM manipulation**.

3. OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript classes and objects.
 - To create a Student class with properties such as Name, Roll Number, Department, and CGPA.
 - To create a Student object using user-provided values.
 - To understand DOM selection and manipulation.
 - To dynamically create and display HTML elements.
 - To implement event handling using a button click.
 - To apply CSS styling to the dynamically generated student profile.
-

4. THEORY

JavaScript is a programming language used to add dynamic and interactive functionality to web pages.

JavaScript Classes provide a blueprint for creating objects with properties and methods. In this application, a Student class is used to store student information such as name, roll number, department, and CGPA.

DOM (Document Object Model) represents the HTML document as a tree of objects, allowing JavaScript to access and modify webpage elements dynamically.

In this application:

- `class Student` is used to define the structure of a student object.
- `constructor()` initializes the student properties.
- `new Student()` creates a Student object using user-provided values.
- `document.getElementById()` is used to select HTML elements from the DOM.
- `.value` retrieves the values entered by the user.
- `addEventListener()` handles the button click event.
- `document.createElement()` dynamically creates new HTML elements.
- `textContent` adds student information to the dynamically created elements.
- `appendChild()` adds the created elements to the webpage.

The basic flow of the application is:

User Input → Button Click → Student Object → DOM Manipulation → Student Profile Display

5. ALGORITHM / PROCEDURE

1. Create an HTML file containing input fields for Name, Roll Number, Department, and CGPA.
 2. Add a **Create Profile** button to the webpage.
 3. Create a JavaScript file and define a Student class.
 4. Define the `constructor()` to initialize Name, Roll Number, Department, and CGPA.
 5. Select the Create Profile button using `document.getElementById()`.
 6. Add a click event listener to the button.
 7. Retrieve the values entered by the user from the input fields.
 8. Create a Student object using the new `Student()` constructor.
 9. Select the profile container from the DOM.
 10. Create HTML elements dynamically using `document.createElement()`.
 11. Display the student details using `textContent`.
 12. Add the dynamically created elements to the profile using `appendChild()`.
 13. Display the generated student profile on the webpage.
 14. Verify the output by entering different student details.
 15. Compare the actual output with the expected output.
-

6. PROGRAM

main.js

```
class Student {
  constructor(name, rollNo, department, cgpa) {
    this.name = name;
    this.rollNo = rollNo;
    this.department = department;
    this.cgpa = cgpa;
  }
}

const button = document.getElementById("createProfile");//dom function to
get the button element with the id "createProfile" and assign it to the
variable "button".

button.addEventListener("click", function () { //event listner is added to
the button element that listens for a click event. When the button is
clicked, the function inside the event listener will be executed.

  const name = document.getElementById("name").value;
  const rollNo = document.getElementById("rollNo").value;
  const department = document.getElementById("department").value;
  const cgpa = document.getElementById("cgpa").value;

  const student = new Student(name, rollNo, department, cgpa);

  const profileContainer = document.getElementById("profileContainer");

  profileContainer.innerHTML = "";

  const profile = document.createElement("div");
  profile.className = "profile";

  const title = document.createElement("h2");
  title.textContent = "Student Profile";

  const nameElement = document.createElement("p");
  nameElement.textContent = `Name : ${student.name}`;
```

```
const rollElement = document.createElement("p");
rollElement.textContent = `Roll No : ${student.rollNo}`;

const departmentElement = document.createElement("p");
departmentElement.textContent = `Department : ${student.department}`;

const cgpaElement = document.createElement("p");
cgpaElement.textContent = `CGPA : ${student.cgpa}`;

profile.appendChild(title);
profile.appendChild(nameElement);
profile.appendChild(rollElement);
profile.appendChild(departmentElement);
profile.appendChild(cgpaElement);

profileContainer.appendChild(profile);
});
```

main.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile</title>
  <link rel="stylesheet" href="main.css">
</head>
<body>

  <div class="container">
    <h1>Student Profile Generator</h1>

    <div class="form">
      <input type="text" id="name" placeholder="Enter Name">
```

```
        <input type="text" id="rollNo" placeholder="Enter Roll
Number">

        <input type="text" id="department" placeholder="Enter
Department">

        <input type="number" id="cgpa" placeholder="Enter CGPA"
step="0.01">

        <button id="createProfile">Create Profile</button>
    </div>

    <div id="profileContainer"></div>
</div>

<script src="main.js"></script>
</body>
</html>
```

main.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    margin: 0;
    padding: 40px;
}

.container {
    width: 500px;
    margin: auto;
    text-align: center;
}

.form {
    background: white;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
```

```
input {
  width: 90%;
  padding: 10px;
  margin: 8px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

button {
  padding: 10px 20px;
  margin-top: 10px;
  border: none;
  border-radius: 5px;
  background-color: #333;
  color: white;
  cursor: pointer;
}

button:hover {
  background-color: #555;
}

.profile {
  margin-top: 25px;
  padding: 20px;
  background-color: white;
  border-radius: 10px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.15);
  text-align: left;
}

.profile h2 {
  text-align: center;
  margin-bottom: 20px;
}

.profile p {
  font-size: 17px;
  margin: 10px 0;
}
```

7. CODE EXPLANATION

Code	Explanation
<code>class Student</code>	Defines a JavaScript class named <code>Student</code> to represent student information.
<code>constructor(name, rollNo, department, cgpa)</code>	Initializes the properties of a <code>Student</code> object.
<code>this.name = name</code>	Stores the student's name in the object.
<code>this.rollNo = rollNo</code>	Stores the student's roll number.
<code>this.department = department</code>	Stores the student's department.
<code>this.cgpa = cgpa</code>	Stores the student's CGPA.
<code>document.getElementById("createProfile")</code>	Selects the Create Profile button from the HTML document.
<code>addEventListener("click", function() {...})</code>	Executes the specified function when the button is clicked.
<code>document.getElementById("name").value</code>	Retrieves the name entered by the user.
<code>document.getElementById("rollNo").value</code>	Retrieves the roll number entered by the user.
<code>document.getElementById("department").value</code>	Retrieves the department entered by the user.
<code>document.getElementById("cgpa").value</code>	Retrieves the CGPA entered by the user.
<code>new Student(name, rollNo, department, cgpa)</code>	Creates a new <code>Student</code> object using the user-provided values.

<code>document.getElementById("profileContainer")</code>	Selects the container where the generated profile will be displayed.
<code>profileContainer.innerHTML = ""</code>	Clears any previously generated student profile.
<code>document.createElement("div")</code>	Dynamically creates a new <code><div></code> element.
<code>profile.className = "profile"</code>	Assigns the CSS class <code>profile</code> to the newly created element.
<code>document.createElement("h2")</code>	Dynamically creates a heading element for the profile title.
<code>document.createElement("p")</code>	Dynamically creates paragraph elements for student details.
<code>textContent</code>	Sets the text content of the dynamically created elements.
<code>appendChild()</code>	Adds the dynamically created elements to the profile.
<code>profileContainer.appendChild(profile)</code>	Adds the completed student profile to the webpage.

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Actual Result
TC-01	Sravan kumar	24126552085	CSM	8.50	Student profile displayed correctly	Student profile displayed correctly
TC-02	Bhuvan teja	24126552084	CSM	9.20	Student profile displayed correctly	Student profile displayed correctly

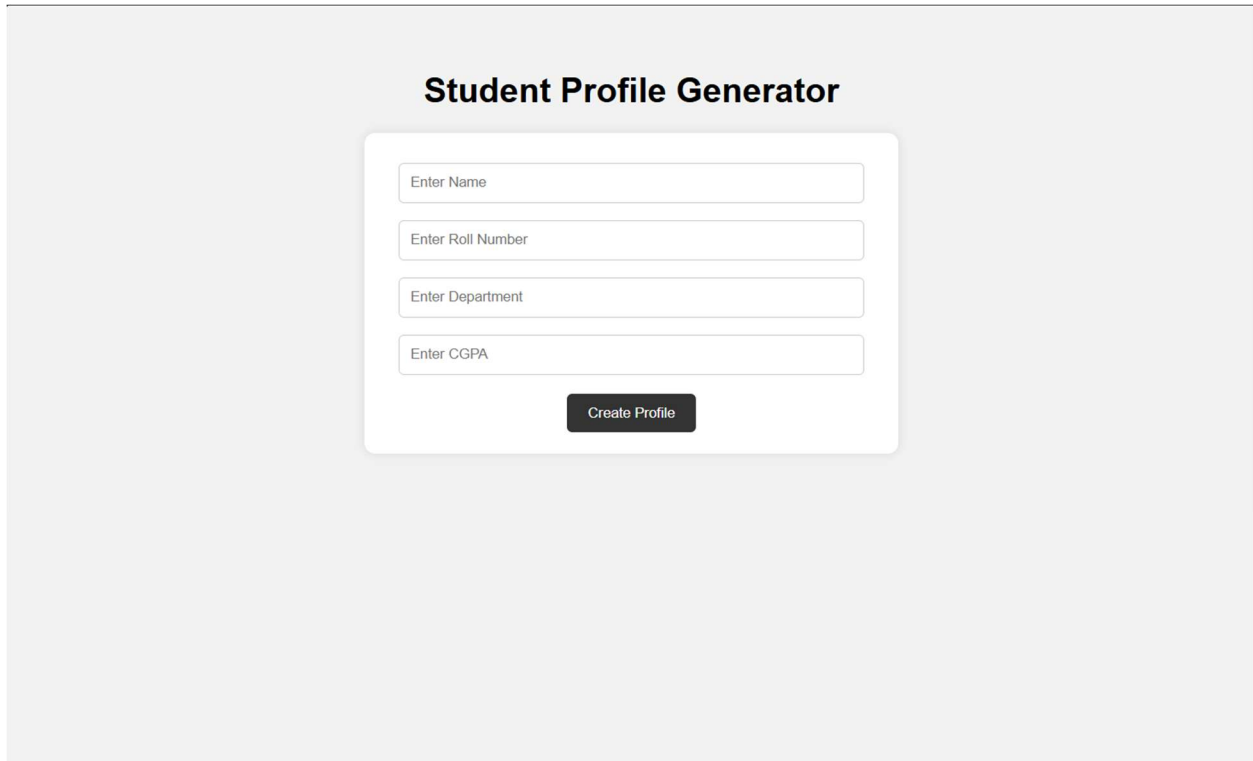
TC-03	Lekhya sri	241265521 23	CSM	9.80	Student profile displayed correctly	Student profile displayed correctly
TC-04	gouri	241265520 76	CSM	9.90	Student profile displayed correctly	Student profile displayed correctly
TC-05	bhanu	241265520 68	CSM	10.00	Student profile displayed correctly	Student profile displayed correctly

Additional Validation Test

Test Case	Input	Expected Result	Actual Result
TC-05	Enter a new set of student details after generating a profile	Previous profile should be cleared and the new profile should be displayed	Previous profile cleared and new profile displayed correctly

9. OUTPUT

Output 1: code execution(before entering data)



The screenshot shows a web form titled "Student Profile Generator" on a light gray background. The form is a white rounded rectangle containing four input fields and a button. The input fields are labeled "Enter Name", "Enter Roll Number", "Enter Department", and "Enter CGPA". Below the fields is a dark gray button with the text "Create Profile".

Student Profile Generator

Enter Name

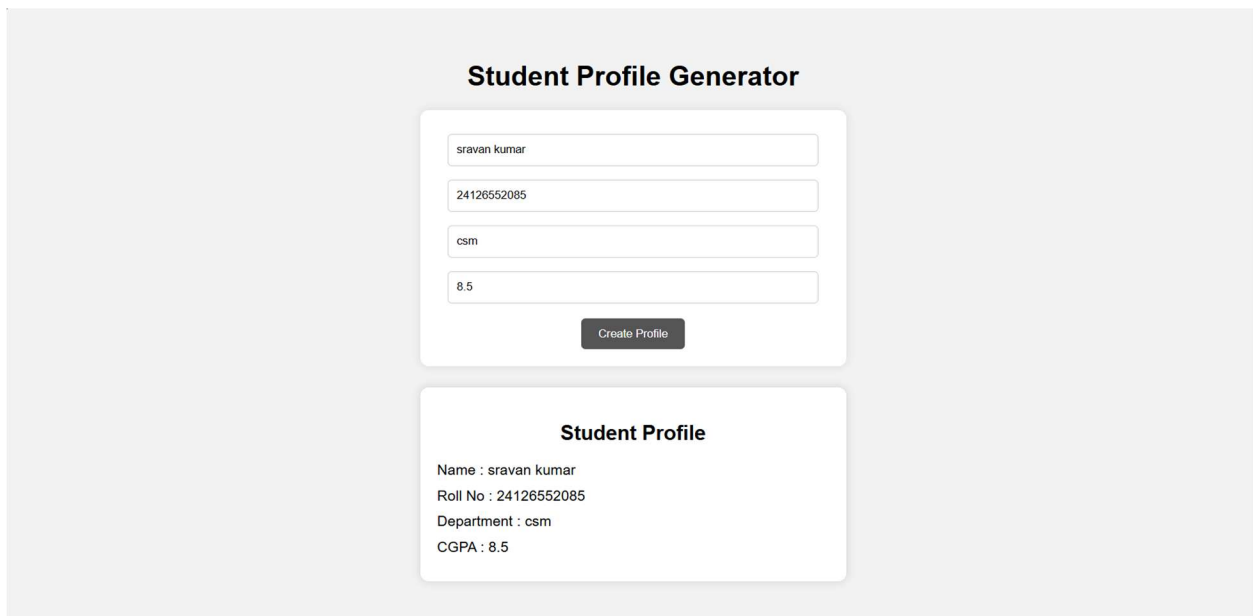
Enter Roll Number

Enter Department

Enter CGPA

Create Profile

Output 2: after filling the form



The screenshot shows the same "Student Profile Generator" form, but now the input fields are filled with data: "sravan kumar", "24126552085", "csm", and "8.5". The "Create Profile" button is still present. Below the form, a new white rounded rectangle titled "Student Profile" displays the entered information in a list format.

Student Profile Generator

sravan kumar

24126552085

csm

8.5

Create Profile

Student Profile

Name : sravan kumar
Roll No : 24126552085
Department : csm
CGPA : 8.5

10. RESULT

Thus, a **Student Profile Generator** was successfully developed using **JavaScript classes and DOM manipulation**. A Student class was created, user-provided student details were used to create objects, and the profile was dynamically generated and displayed on the webpage. The application was successfully verified using multiple test cases.
